

# Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Generici I

a cura di Massimo La Morgia



SAPIENZA  
UNIVERSITÀ DI ROMA

\*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

\*\*I crediti sulle slide di questo corso sono riportati nell'ultima slide

**equals Vs. hashCode**

# equals Vs. hashCode

- $a.equals(b) \Rightarrow a.hashCode() == b.hashCode()$
- $a.hashCode() == b.hashCode() \not\Rightarrow a.equals(b)$

# Esempio

```
public class Persona {  
  
    private String nome;  
    private String cognome;  
  
    public Persona(String nome, String cognome)  
    {  
        this.nome=nome;  
        this.cognome=cognome;  
    }  
  
    // ....  
}
```

---



# equals

```
-  
// ....  
@Override  
public boolean equals(Object obj) {  
    if (this == obj)  
        return true;  
    if (obj == null)  
        return false;  
    if (getClass() != obj.getClass())  
        return false;  
    Persona other = (Persona) obj;  
    if (cognome == null) {  
        if (other.cognome != null)  
            return false;  
    } else if (!cognome.equals(other.cognome))  
        return false;  
    if (nome == null) {  
        if (other.nome != null)  
            return false;  
    } else if (!nome.equals(other.nome))  
        return false;  
    return true;  
}
```

# hashCode

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + ((cognome == null) ? 0 : cognome.hashCode());
    result = prime * result + ((nome == null) ? 0 : nome.hashCode());
    return result;
}
```

# Tipi Generici parte I

# Che cosa accomuna queste due classi?

```
public class CoppiaDiInteri
{
    private int a, b;

    public CoppiaDiInteri(int a, int b)
    {
        this.a = a;
        this.b = b;
    }

    public int getPrimo() { return a; }
    public int getSecondo() { return b; }
}
```

```
public class CoppiaDiDouble
{
    private double a, b;

    public CoppiaDiDouble(double a, double b)
    {
        this.a = a;
        this.b = b;
    }

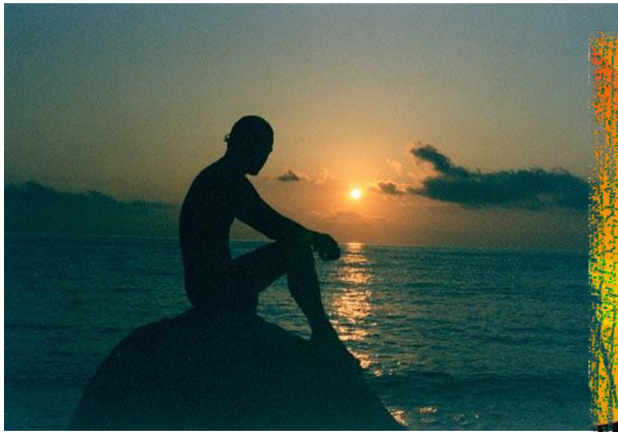
    public double getPrimo() { return a; }
    public double getSecondo() { return b; }
}
```

Stesso codice,  
ma tipi dei campi  
diversi!

Ci vorrebbe  
qualcosa di  
**generico**...

# Che cos'è un tipo generico?

- I tipi generici, in Java, sono un **modello di programmazione** che permette di definire, con una sola dichiarazione, un intero **insieme di metodi o di classi**
- Un meccanismo **MOLTO** potente
- Da usare con **consapevolezza**



Cup<T>

# Avete già utilizzato ampiamente i generici!

- Tutte le collezioni di Java sono generiche
- **Creare istanze** di classi con generici dove i

parametri generici sono sostituiti da tipi reali :

```
new ArrayList<String>();
```

- **Dichiarare e assegnare variabili** di tipi generici dove i parametri generici sono sostituiti da tipi reali

```
List<String> listaDiStringhe = new ArrayList<String>();
```

Posso anche scrivere:  
`new ArrayList<>();`

- **Dichiarare metodi** che prendono in input tipi generici dove i parametri generici sono sostituiti da tipi reali

```
public void metodo(List<String> lista)
{
    // ...
}
```

# Esempio di classe generica

```
public class Valore<T>
{
    private final T val;

    public Valore(T val) { this.val = val; }

    public T get() { return val; }

    @Override
    public String toString() { return ""+val; }

    public String getType() { return val.getClass().getName(); }
}
```

Definisce un tipo generico della classe

Usa il tipo generico della classe

- Per definire un tipo generico della classe, si utilizza la sintassi a **parentesi angolari** dopo il nome della classe con il tipo generico da utilizzare
- Da quel punto, si utilizza il tipo generico come un qualsiasi altro tipo di classe



# Istanziare la classe generica

Inferenza automatica del  
tipo generico

```
public static void main(String[] args)
{
    Valore<Integer> i = new Valore<>(42);
    Valore<String> s = new Valore<>("ciao");
    Valore<Valore<String>> v = new Valore<>(s);

    System.out.println(i.get()+":"+i.getType());
    System.out.println(s.get()+":"+s.getType());
    System.out.println(v.get()+":"+v.getType());
}
```

- L'output sarà:

```
42:java.lang.Integer
ciao:java.lang.String
ciao:Valore
```

```
public class Valore<T>
{
    private final T val;

    public Valore(T val) { this.val = val; }

    public T get() { return val; }

    @Override
    public String toString() { return ""+val; }

    public String getType() { return val.getClass().getName(); }
}
```

## Altro esempio: Coppia di elementi di tipo generico

```
public class Coppia<T>
{
    private T a, b;

    public Coppia(T a, T b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public T getSecondo() { return b; }
}
```

# Esempio: utilizzare la classe Coppia

- Semplicemente si istanzia la classe specificando il tipo desiderato:

```
Coppia<Integer> ci = new Coppia<Integer>(10, 20);  
Coppia<Double> cd = new Coppia<Double>(10.5, 20.5);  
Coppia<String> cs = new Coppia<String>("abc", "def");  
Coppia<Object> co = new Coppia<Object>("abc", 20);
```

- ...e chi più ne ha più ne metta!

```
public class Coppia<T>  
{  
    private T a, b;  
  
    public Coppia(T a, T b)  
    {  
        this.a = a;  
        this.b = b;  
    }  
  
    public T getPrimo() { return a; }  
    public T getSecondo() { return b; }  
}
```

# Specificare più tipi generici di classe

- Specifichiamo i tipi generici separati da virgola:

```
public class Coppia<T, S>
{
    private T a;
    private S b;

    public Coppia(T a, S b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public S getSecondo() { return b; }
}
```

- Per convenzione i tipi generici sono chiamati con le lettere T, S, ecc. (E nel caso in cui siano elementi di una collection)

# I generici funzionano solo con i tipi derivati

- Non è possibile utilizzare tipi primitivi, ad es. `int`, `double`, `char`, ecc.
- Es:

```
Valore<int> v;  
Valore<double> v;
```

# I tipi generici differiscono sulla base dei tipi reali.

- Ad esempio, `Valore<Integer>` non è compatibile con `Valore<String>`:

```
public static void main(String[] args)
{
    Valore<Integer> i = new Valore<>(42);
    Valore<String> s = new Valore<>("ciao");
    Valore<Valore<String>> v = new Valore<>(s);
```

```
    s = i;
```

Type mismatch: cannot convert from Valore<Integer> to Valore<String>

2 quick fixes available:

➤ [Change type of 's' to 'Valore<Integer>'](#)

➤ [Change type of 'i' to 'Valore<String>'](#)

Press 'F2' for focus

# Estendere le classi generiche

- Ovviamente è possibile estendere le classi generiche per creare classi più specifiche
- Ad esempio, una classe **Orario** può estendere la classe **Coppia**:

```
public class Orario extends Coppia<Integer, Integer>
{
    public Orario(Integer a, Integer b)
    {
        super(a, b);
    }
}
```

- O una classe **Data**:

```
public class Data extends Coppia<Integer, Coppia<Integer, Integer>>
{
    public Data(Integer giorno, Integer mese, Integer anno)
    {
        super(giorno, new Coppia<Integer, Integer>(mese, anno));
    }
}
```



# Esercizio: Lista Linkata generica

- Progettare una classe generica `ListaLinkata`

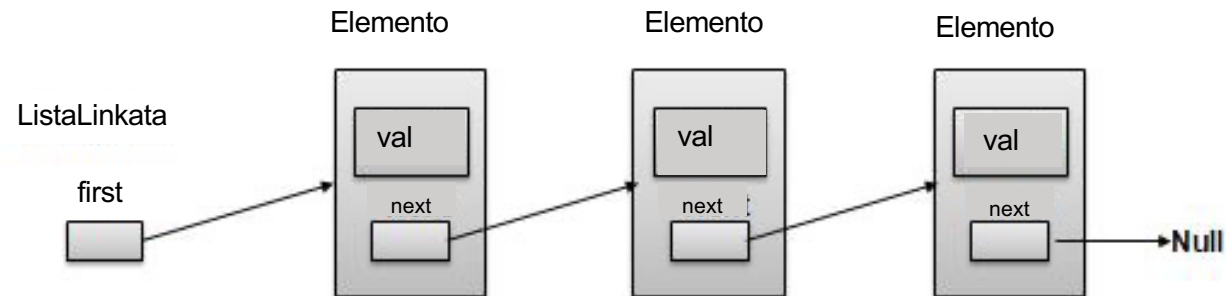
```
public class ListaLinkata<T>
{
    private Elemento first;

    private class Elemento
    {
        private T val;
        private Elemento next;
        public Elemento(T val, Elemento next) { this.val = val; this.next = next; }
    }

    public void addFirst(T e)
    {
        first = new Elemento(e, first);
    }

    public void removeFirst()
    {
        first = first.next;
    }

    public String toString()
    {
        StringBuffer sb = new StringBuffer();
        Elemento e = first;
        while(e != null) { sb.append(e.val); if (e.next != null) sb.append(","); e = e.next; }
        return sb.toString();
    }
}
```







## Esercizio: la classe Pila

- Progettare una classe **generica Pila** implementata mediante lista di elementi (provate anche con l'array)
- La classe è costruita con la dimensione iniziale dell'array ed implementa i seguenti metodi:
  - **push**: inserisce un elemento in cima alla pila
  - **pop**: elimina e restituisce l'elemento in cima alla pila
  - **peek**: restituisce l'elemento in cima alla pila
  - **isEmpty**: restituisce true se la pila è vuota

# Le interfacce Comparable e Comparator sono generiche

```
public interface Comparable<T>
{
    int compareTo(T o);
}
```

```
public interface Comparator<T>
{
    int compare(T o1, T o2);
}
```

## Esempio: estendere un'interfaccia generica con vincolo di comparabilità sul tipo generico

```
public interface MinMax<T extends Comparable<T>>
{
    T min();
    T max();
}
```

```
public class MyClass<T extends Comparable<T>> implements MinMax<T>
{
    // ...
}
```

**<T extends S>**

permette di vincolare un simbolo generico T ad essere di tipo S o una sottoclasse di S

Quali di queste estensioni sono corrette?

`class MyClass<T extends Comparable<T>>  
implements MinMax<T extends Comparable<T>> { /* ...  
*/ }` 

`class MyClass implements MinMax<T> { /* ... */ }` 

`class MyClass implements MinMax<Integer> { /* ... */ }` 

# Generici e collezioni (1)

- Alcuni esempi prototipici:

```
public interface List<E>
```

```
{
```

```
    void add(E x);
```

```
    Iterator<E> iterator();
```

```
}
```

```
public interface Iterator<E>
```

```
{
```

```
    E next();
```

```
    boolean hasNext();
```

```
    default void remove();
```

```
}
```

## Generici e collezioni (2)

- L'utilizzo principale dei tipi generici è nelle collezioni
- Ad esempio, vediamo come è definita `ArrayList`:

```
public class ArrayList<E>  
    extends AbstractList<E>  
    implements List<E>, RandomAccess, Cloneable, Serializable
```

- Quando scriviamo:

```
new ArrayList<String>();
```

- La classe viene trattata «come»:

```
public class ArrayList<String> extends AbstractList<String>  
implements List<String>, RandomAccess, Cloanable,  
Serializable
```

# Definire un metodo generico

- Per definire un metodo generico con proprio tipo generico è necessario anteporre il tipo generico (o l'elenco) tra parentesi angolari al tipo di ritorno:

Si deve dichiarare l'elenco dei simboli generici utilizzati dal metodo

```
static public <T> void esamina(ArrayList<T> lista)
{
    for (T o : lista)
        System.out.println(o.toString());
}
```

Perché posso chiamare toString() su un tipo generico?



## Esercizio: Inverti lista e massimo

- Implementare i seguenti due metodi generici statici:
  - **inverti**, che data in input una lista di elementi di tipo generico, restituisca un'altra lista con gli elementi in ordine invertito
  - **max**, che data in input una lista di elementi di tipo generico, ne restituisca il valore massimo
- **Nota:** per il secondo metodo è necessario utilizzare il costrutto `<T extends InterfacciaOClasse>`, che impone un vincolo sul supertipo di T



# Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.